

PRACTICAL-4

AIM: Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce

PROBLEM DEFINITION:

A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce.

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records.

Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs.

Task 3: Develop and implement the Reducer program to determine the maximum temperature recorded for each year.

Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS. Verify the correctness of the output generated.

Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset. Discuss the advantages of using MapReduce for large-scale climate data analysis.

Supplementary Problem: Find the minimum temperature recorded for each year. Calculate average yearly temperatures. Identify the year with the highest recorded temperature overall.

CODE (SUPPLEMENTARY PROBLEMS INCLUDED):

1. mapper.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
for line in sys.stdin:
    line = line.strip()
```

```
try:
    if not line:
        continue

    year = line[15:19]
    temperature = int(line[87:92])
    quality = line[92]

    if temperature == 9999:
        continue

    print(f'{year}\t{temperature}')

except (ValueError, IndexError):
    continue
```

2. reducer.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_year = None
```

```
maximum_temperature = None
```

```
for line in sys.stdin:
```

```
    line = line.strip()
```

```
    try:
```

```
        year, temperature = line.split("\t")
```

```
        temperature = int(temperature)
```

```
        if current_year == year:
```

```
            if temperature > maximum_temperature:
```

```
                maximum_temperature = temperature
```

```
        else:
```

```
            if current_year is not None:
```

```
                print(f'{current_year}\t{maximum_temperature}')
```

```
        current_year = year
```

```
        maximum_temperature = temperature
```

```
except ValueError:  
    continue
```

```
if current_year is not None:  
    print(f"{current_year}\t{maximum_temperature}")
```

3. min_mapper.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
for line in sys.stdin:  
    line = line.strip()
```

```
try:  
    if not line:  
        continue
```

```
    year = line[15:19]  
    temperature = int(line[87:92])
```

```
    if temperature == 9999:  
        continue
```

```
    print(f"{year}\t{temperature}")
```

```
except (ValueError, IndexError):  
    continue
```

4. min_reducer.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_year = None  
minimum_temperature = None
```

```
for line in sys.stdin:  
    line = line.strip()
```

```
try:
    year, temperature = line.split("\t")
    temperature = int(temperature)

    if current_year == year:
        if temperature < minimum_temperature:
            minimum_temperature = temperature
    else:
        if current_year is not None:
            print(f'{current_year}\t{minimum_temperature}')

        current_year = year
        minimum_temperature = temperature

except ValueError:
    continue
```

```
if current_year is not None:
    print(f'{current_year}\t{minimum_temperature}')
```

5. avg_reducer.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_year = None
total_temperature = 0
count = 0
for line in sys.stdin:
    line = line.strip()
```

```
try:
    year, temperature = line.split("\t")
    temperature = int(temperature)

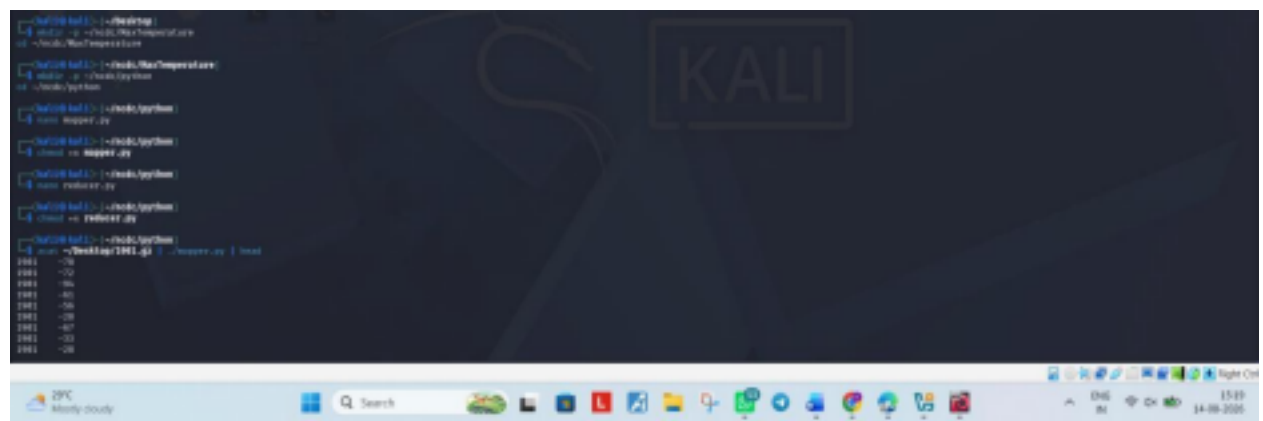
    if current_year == year:
        total_temperature += temperature
        count += 1
```

if current_year is not None:

```
current_year = year
total_temperature = temperature
count = 1
```

```
if current_year is not None:
    average = total_temperature / count
    print(f'{current_year}\t{average:.2f}')
```

[illegible]



[illegible]

KEY QUESTIONS / ANALYSIS:

1) How is weather data parsed and processed using MapReduce?

Weather data is stored in HDFS as large input files. The Mapper reads each weather record, extracts the year and temperature, and generates key-value pairs. Hadoop groups all temperature values having the same year and sends them to the Reducer. The Reducer compares the temperatures for each year and selects the maximum temperature. Finally, the yearly maximum temperatures are stored back in HDFS.

2) What key-value pairs are generated by the Mapper?

The Mapper generates the year as the key and the temperature as the value.

Format:

(year, temperature)

Example:

2020 35

2020 38

2021 36

2021 40

Here, 2020 and 2021 are keys, while the temperature values are the corresponding values.

3) How does the Reducer identify the maximum temperature for a given year?

The Reducer receives all temperature values belonging to the same year. It compares these values one by one and keeps track of the highest value.

Example:

2020 → [35, 38, 36, 40]

The Reducer selects:

2020 → 40

Thus, the output contains the maximum temperature recorded for each year.

4) What challenges arise when processing large climate datasets?

Some major challenges are:

1. Huge data volume – Climate datasets can contain billions of records.
2. Data cleaning – Missing, invalid, or inconsistent weather records may occur.
3. Processing time – Sequential processing can take a long time.
4. Data distribution – Data must be efficiently distributed across multiple nodes.
5. Fault tolerance – Hardware or node failures can occur during processing.
6. Storage management – Large datasets require significant storage capacity.

5) How does Hadoop improve scalability and performance for weather data analytics?

Hadoop improves scalability and performance by distributing large datasets across multiple machines in a cluster. HDFS provides distributed storage, while MapReduce processes data in parallel across multiple nodes. If one node fails, Hadoop can recover the data using replication and rerun failed tasks. Therefore, Hadoop can efficiently process very large climate datasets with better scalability, fault tolerance, and performance. It also reduces processing time by dividing complex tasks into smaller tasks that can run simultaneously. Hadoop can easily scale by adding more nodes to the cluster as the size of the dataset increases.

CONCLUSION:

The practical successfully demonstrated how Hadoop MapReduce can be used to process large scale NCDC weather data and find the maximum temperature recorded for each year. The Mapper extracted year-temperature key-value pairs, while the Reducer grouped the data and identified the maximum temperature for each year. This practical also showed how MapReduce can be extended to find minimum and average temperatures. Overall, Hadoop provides efficient, scalable, parallel, and fault-tolerant processing for large climate datasets.